

Failų medis

```
.
├─ build_indexes
├─ central/
│   ├── Dockerfile
│   ├── main.py
│   └─ requirements.txt
├─ data/
├─ docker-compose.yml
├─ start_workers
├─ stop_workers
└─ worker/
    ├── Dockerfile
    ├── main.py
    └─ requirements.txt
```

start_workers, stop_workers – lokaliai darbinį mazgą paleidimui ir stabdymui

build_indexes – MMSeqs2 indeksavimo paleidimas (reikalinga dėl [problemų](#))

Užklauso kelias

1. Vartotojas siunčia užklausa su DNR seka POST metodu į centrinį mazgą
2. Centrinis mazgas ištransliuoja seką visiems darbiniam mazgams async HTTP užklausomis
3. Kiekvienas darbinis mazgas perskaito užklausa, konvertuoja ją į MMSeqs2 vidinį formatą ir lygina ją su turimais duomenimis
4. Kiekvienas darbinis mazgas grąžina po sąrašą pilnų sekų, kurios pakankamai gerai atitiko prieš tai gautą užklausa
5. Centriniam mazgui sulaukus visų darbinių atsako, jis sukuria laikiną FASTA failą su visais sulauktais kandidatais, dedupliuoja
6. Su FASTA failu paleidžiamas pilnas blastn tarp kandidatų, sukurama statistika
7. Centrinis mazgas grąžina JSON su geriausiais rezultatais, kiek laiko viskas užtruko, gražiai suformatuota ataskaita.

Sistemos paleidimas

Yra du būdai paleisti sistemą – paprastas ir Docker

Paleidimui reikalingi duomenys – data/ direktorijoje į shard1, shard2, shard3, shard4 suskirstyti FASTA failai.

Pavyzdinius duomenis galima paruošti taip:

```
wget -P data/  
ftp://ftp.sra.ebi.ac.uk/vol1/fastq/SRR258/003/SRR2584863/SRR2584863\_1.fastq.gz  
(~175mb)
```

Konvertuojame FASTQ į FASTA

```
zcat data/SRR2584863_1.fastq.gz | head -1600000 | awk 'NR%4==1{print  
">"substr($0,2)} NR%4==2{print}' > data/all_reads.fasta
```

Padilinimas į 4 dalis po 100,000

```
python3 -c "  
total_reads = 400000  
per_shard = total_reads // 4  
outfiles = [open(f'data/shard_{i+1}.fasta', 'w') for i in range(4)]  
read_count = 0  
with open('data/all_reads.fasta') as f:  
    for line in f:  
        if line.startswith('>'):  
            shard_idx = min(read_count // per_shard, 3)  
            read_count += 1  
            outfiles[shard_idx].write(line)  
for f in outfiles: f.close()"
```

MMseqs2 indeksų sukūrimas (kol kas daromas ne izoliuotai
dockeryje, nes trūksta kompiuterio resursų)
./build_indexes

Paprastasis paleidimas:

```
sudo apt install mmseqs2 ncbi-blast+  
python3 -m venv .venv  
source .venv/bin/activate  
pip install -r worker/requirements.txt  
pip install -r central/requirements.txt  
./start_workers  
Kitame terminale:  
uvicorn central.main:app --port 8080
```

Paleidimas su Docker:

```
docker compose up  
(Mano kompiuteryje Docker buvo priskirta 12GB RAM.)
```

Sistemos vidinė komunikacija

Request laukeliai:

- `sequence` – seka, pagal kurią ieškoma
- `query_id` - užklauso ID
- `top_n` – neprivalomas argumentas, nurodantis kiek rezultatų norime gauti atsake

Response laukeliai:

- `query_id` - u-klausos ID
- `query_length` - bazių porų kiekis užklausoje
- `workers_queried` - darbinių mazgų veikimo kiekis
- `worker_stats` - kiekvieno darbinio mazgo statistika
- `candidates_after_dedup` – suma visų darbinių mazgų grąžintų rezultatų
- `hits` - BLAST rezultatas
- `report` – žmogui skirta ataskaita
- `total_pipeline_time_seconds` – sekundžių kiekis tarp užklauso gavimo ir šio atsako

request pavzdys:

```
curl -X POST http://localhost:8080/query \
-H "Content-Type: application/json" \
-d "{
  \"sequence\": \"$(sed -n '2p' data/shard_1.fasta)\",
  \"query_id\": \"real_test\",
  \"top_n\": 3
}"
```

response pavyzdys:

```
{
  "query_id": "real_test",
  "query_length": 150,
  "workers_queried": 4,
  "worker_stats": [
    {"worker_url": "http://worker1:8000", "candidates_found": 7,
"search_time_seconds": 99.771},
    {"worker_url": "http://worker2:8000", "candidates_found": 5,
"search_time_seconds": 99.767},
    {"worker_url": "http://worker3:8000", "candidates_found": 2,
"search_time_seconds": 99.739},
    {"worker_url": "http://worker4:8000", "candidates_found": 4,
"search_time_seconds": 99.753}
  ],
  "candidates_after_dedup": 17,
  "hits": [...],
  "report": "=====
DISTRIBUTED BLAST PIPELINE REPORT
=====
Query ID:      real_test
Query length:  150 bp
Nodes queried: 4

WORKER STATISTICS
-----
http://worker1:8000: 7 candidates (99.771s)
http://worker2:8000: 5 candidates (99.767s)
http://worker3:8000: 2 candidates (99.739s)
http://worker4:8000: 4 candidates (99.753s)

Total candidates after deduplication: 17

TOP 3 HITS (sorted by E-value)
-----
1. SRR2584863.1
  Identity: 100.0% | Length: 150 | E-value: 6.23e-79 | Score: 278.0
2. SRR2584863.377504
  Identity: 100.0% | Length: 148 | E-value: 8.06e-78 | Score: 274.0
3. SRR2584863.140792
  Identity: 100.0% | Length: 138 | E-value: 2.92e-72 | Score: 255.0

NOTE: E-values are relative to the candidate pool size, not the full
dataset. Rankings are valid but absolute values are not comparable to
standard full-database BLAST results.
=====",
  "total_pipeline_time_seconds": 100.501
}
```

Pirminio filtro pasirinkimas darbinuose mazguose, svarstyto alternatyvos

- Minimap2 - palaiko reikiamas paieškas, bet labiau skirtas ilgų sekų išlyginimui. Orientuotas į tai, kur seka genome, o ne kokia tai seka duomenų bazėje.
- grep/grepq – nors labai lengvi, neturi pakankamai lankstumo, ypač su insercijomis, delecijomis.
- BLAST – didelis, lėtas.
- MMseqs2 – lengvas, lankstus, itin greitas.

Bandomieji duomenys

Naudoti E. coli genomo duomenys (accession SRR2584863). 150Bp sekos, jų buvo apie 1.5 milijono, panaudota 400 tūkstančių. Jie paskirstyti per 4 darbinius mazgus po 100 tūkstančių.

Žinomos problemos ir patobulinimai

Problemos

- Jei MMSeqs2 createindex paleidžiamas Docker aplinkoje, sistema pradeda lūžinėti dėl atminties trūkumo.
- Docker Desktop aplinkoje vienos užklausos apdorojimas užtrunka apie 100s, Docker nenaudojant apie 16s. Rezultatai identiški
- E-value reikšmės iš galutinio BLAST yra neįtikėtinai geros. Taip nutinka, nes lyginami tik pirminio filtro atsijoti kandidatai.
- Kodas asynchroniškas, tačiau nėra apsaugų nuo vartotojų, kurie galėtų siųsti daug užklausų ir greitai nulaužti sistemą.

Svarstomi patobulinimai antrai versijai

- Nereikalausti vartotojo rankinio FASTQ → FASTA konvertavimo. Daryti tai pagal poreikį dinamiškai kode.
- Pirmoje versijoje jau galima paleisti kitokį kiekį darbinių mazgų negu 4 nustatant aplinkos kintamąjį WORKERS, tačiau reiktų sugalvoti sprendimą su duomenų užpildymu, kadangi dabar viskas paruošta lygiai keturiems.
- Automatiniai testai su Pytest.
- Paprasta WEB sąsaja. Jau naudojamas FastAPI, todėl tai nebūtų sunku.
- Pabandyti Docker be Docker Desktop.
- Diagrama dokumentacijai